

README Structure

1. Overview

This repository provides a set of scripts to automate the installation and validation of both the execution environment and the application.

The workflow is designed to follow a simple sequence:

- Prepare the execution environment.
- Validate the environment.
- Install the application.
- Validate the application.

2. Application Architecture

Provide a high level overview of the application's architecture.

Insert the application architecture diagram.

Example:

`docs/images/application-architecture.png`

Include a brief description of the main components, their responsibilities, and their interactions.

3. Network Architecture

Provide an overview of the network topology required to deploy the application.

Insert the network architecture diagram.

Example:

`docs/images/network-architecture.png`

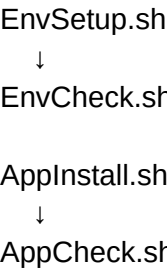
Describe:

- Network zones
- Servers and services
- Communication flows
- Required ports and protocols
- External dependencies

4. Script Overview

Script	Purpose	Automatically Runs
EnvSetup.sh	Installs and configures the execution environment	EnvCheck.sh
EnvCheck.sh	Verifies that the execution environment is correctly configured	None
AppInstall.sh	Installs and configures the application from the source code	AppCheck.sh
AppCheck.sh	Verifies that the application has been installed correctly	None

Workflow:



5. Prerequisites

Supported Operating Systems

List all supported operating systems, for example:

- Ubuntu 22.04
- Ubuntu 24.04
- Debian 12
- Rocky Linux 9
- Windows (WSL)
- macOS

Required Dependencies

List all required tools, for example:

- Bash
- Git
- Curl
- Wget
- Python
- Docker

References

Provide links to any required software or installation guides.

6. Environment Installation (**EnvSetup.sh**)

Purpose

EnvSetup.sh prepares the execution environment by installing and configuring all required components.

At the end of the installation, the script automatically launches **EnvCheck.sh** to validate the environment.

Usage

```
chmod +x EnvSetup.sh  
./EnvSetup.sh
```

Installation Steps

1. Verify execution permissions.
2. Detect the operating system.
3. Install required packages.
4. Configure the execution environment.
5. Configure environment variables.
6. Perform cleanup if required.
7. Run the environment validation.

Expected Result

Environment installation completed.

Launching environment validation...

7. Environment Validation (**EnvCheck.sh**)

Purpose

EnvCheck.sh verifies that the execution environment is fully operational.

Validation Checks

Typical checks include:

- Operating system compatibility
- Bash version
- Git installation
- Docker availability
- Python version
- Environment variables
- File permissions

Validation Output

Display the status of every validation check.

Example:

```
[PASS] Operating System
[PASS] Git
[PASS] Docker
[PASS] Python
[PASS] Environment Variables
[PASS] Permissions
```

8. Application Installation (**AppInstall.sh**)

Purpose

AppInstall.sh installs and configures the application after the source code has been retrieved.

Once the installation is complete, the script automatically launches **AppCheck.sh**.

Prerequisites

- Environment successfully validated
- Source code available

Usage

```
chmod +x AppInstall.sh
./AppInstall.sh
```

Installation Steps

1. Verify prerequisites.
2. Install application dependencies.
3. Build the application if required.
4. Apply configuration.
5. Initialize required resources.
6. Generate required files.
7. Run the application validation.

Expected Result

Application installation completed.

Launching application validation...

9. Application Validation (**AppCheck.sh**)

Purpose

AppCheck.sh verifies that the application is correctly installed and ready for use.

Validation Checks

Typical checks include:

- Required dependencies
- Configuration files
- Database connectivity
- Running services
- API availability
- File permissions

Validation Output

Display the status of every validation check.

Example:

```
[PASS] Configuration
[PASS] Database
[PASS] Services
[PASS] API
```

10. Docker Deployment

Docker Image

Specify the Docker image to use.

Example:

```
my-application:latest
```

Environment Variables

List the required environment variables.

Variable	Description	Required	Default
APP_ENV	Application environment	Yes	production
APP_PORT	Application port	No	8080

DB_HOST	Database host	Yes	N/A
---------	---------------	-----	-----

Run the Container

Provide the command to start the application.

```
docker run \
--name my-application \
-p 8080:8080 \
--env-file .env \
my-application:latest
```

Or, if a `docker-compose.yml` is provided, means if you have docker compose yaml example, put it there then show how to use it :

```
docker compose up -d
```

11. Troubleshooting

Provide common issues, possible causes, and recommended solutions.

Typical examples include:

- Missing dependency
- Unsupported operating system
- Permission issues
- Missing environment variables
- Service startup failures

12. References

Include links to:

- Project documentation
- Dependency documentation
- Installation guides
- External tools and resources